Register Number:192425325

Name:B.Archana

CO4-AT-1

MLA0201-Fundamentals Of Machine Learning

Coding Challenge

Question 1: Bayesian Network for Disease Prediction

Aim

To construct a **Bayesian Network** for predicting **Diabetes** based on **Obesity** and **High Blood Sugar**, define suitable Conditional Probability Tables (CPTs), and perform probabilistic inference to determine the probability of Diabetes when both symptoms are present.

Kaggle link: https://www.kaggle.com/datasets/aastik1844/diabetes-dataset

Dataset Name: Diabetes Dataset

Procedure

1. Import the required Python libraries.
2. Load the diabetes dataset from the Kaggle input directory.
3. Construct a Bayesian Network with:
    - **Obesity → Diabetes**
    - **High Blood Sugar → Diabetes**
4. Define the CPT for Obesity.
5. Define the CPT for High Blood Sugar.
6. Define the CPT for Diabetes based on Obesity and High Blood Sugar.
7. Add all CPTs to the Bayesian Network.
8. Check whether the Bayesian Network is valid.
9. Use **Variable Elimination** for probabilistic inference.
10. Give the evidence:

- Obesity = Yes
- High Blood Sugar = Yes

11. Calculate and display the probability of Diabetes.
12. Display the final prediction.

Bayesian Network Structure

Obesity

↓

Diabetes

↑

High Blood Sugar

Code:

```
# QUESTION 1: BAYESIAN NETWORK FOR DIABETES PREDICTION

# Install pgmpy
!pip install -q pgmpy

# Import libraries
import pandas as pd
from google.colab import files
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# STEP 1: CHOOSE AND UPLOAD CSV FILE
print("Please choose the Diabetes CSV file")
uploaded = files.upload()

# Get uploaded file name
file_name = list(uploaded.keys())[0]

# Read CSV file
data = pd.read_csv(file_name)

print("\n========== DATASET ==========")
print(data.head())

# STEP 2: CREATE BAYESIAN NETWORK
model = DiscreteBayesianNetwork([
    ("Obesity", "Diabetes"),
    ("High_Blood_Sugar", "Diabetes")
])

# STEP 3: CPT FOR OBESITY
cpd_obesity = TabularCPD(
    variable="Obesity",
    variable_card=2,
    values=[
        [0.60],
        [0.40]
    ],
    state_names={
        "Obesity": ["No", "Yes"]
    }
)

# STEP 4: CPT FOR HIGH BLOOD SUGAR
cpd_sugar = TabularCPD(
    variable="High_Blood_Sugar",
    variable_card=2,
    values=[
        [0.70],
        [0.30]
    ],
    state_names={
        "High_Blood_Sugar": ["No", "Yes"]
    }
)
```

```python
# STEP 5: CPT FOR DIABETES
cpd_diabetes = TabularCPD(
    variable="Diabetes",
    variable_card=2,
    values=[
        [0.95, 0.70, 0.60, 0.20],
        [0.05, 0.30, 0.40, 0.80]
    ],
    evidence=["Obesity", "High_Blood_Sugar"],
    evidence_card=[2, 2],
    state_names={
        "Diabetes": ["No", "Yes"],
        "Obesity": ["No", "Yes"],
        "High_Blood_Sugar": ["No", "Yes"]
    }
)

# STEP 6: ADD CPTs
model.add_cpds(
    cpd_obesity,
    cpd_sugar,
    cpd_diabetes
)

# STEP 7: CHECK MODEL
print("\n========== MODEL CHECK ==========")
print("Model Valid:", model.check_model())

# STEP 8: PERFORM INFERENCE
inference = VariableElimination(model)

result = inference.query(
    variables=["Diabetes"],
    evidence={
        "Obesity": "Yes",
        "High_Blood_Sugar": "Yes"
    }
)

# STEP 9: DISPLAY OUTPUT
print("\n======================================")
print("       DIABETES PREDICTION")
print("======================================")

print("\nEvidence:")
print("Obesity = Yes")
print("High Blood Sugar = Yes")

print("\nProbability Distribution:")
print(result)

probability = result.values[1]

print(
    f"\nPredicted Probability of Diabetes = "
    f"{probability * 100:.2f}%"
)

if probability >= 0.5:
    print("Final Prediction: DIABETES = YES")
else:
    print("Final Prediction: DIABETES = NO")
```

Output:

```
Please choose the Diabetes CSV file
Choose Files diabetes (1).csv
diabetes (1).csv(text/csv) - 23873 bytes, last modified: 8/24/2026 - 100% done
Saving diabetes (1).csv to diabetes (1) (1).csv

========== DATASET ==========
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

========== MODEL CHECK ==========
Model Valid: True

======================================
       DIABETES PREDICTION
======================================

Evidence:
Obesity = Yes
High Blood Sugar = Yes

Probability Distribution:
+---------------+-----------------+
| Diabetes      |   phi(Diabetes) |
+===============+=================+
| Diabetes(No)  |          0.2000 |
+---------------+-----------------+
| Diabetes(Yes) |          0.8000 |
+---------------+-----------------+

Predicted Probability of Diabetes = 80.00%
Final Prediction: DIABETES = YES
```

Result

The Bayesian Network successfully predicted the probability of Diabetes. When both Obesity and High Blood Sugar are present, the predicted probability of Diabetes is 80%.

Question 2: Hidden Markov Model for Weather Prediction

Aim

To implement a **Hidden Markov Model (HMM)** for weather prediction using **Sunny, Cloudy, and Rainy** weather states, define transition and emission probabilities, and predict the hidden weather states for a given sequence of observations.

Dataset

File Name: weather_hmm_dataset.csv

Procedure

1. Import the required Python libraries.

2. Upload the weather CSV using **Choose Files**.
3. Read the observation sequence from the dataset.
4. Define the hidden weather states:
    - Sunny
    - Cloudy
    - Rainy
5. Define the transition probability matrix.
6. Define the emission probability matrix.
7. Define the initial state probabilities.
8. Create the Hidden Markov Model.
9. Apply the **Viterbi algorithm** to find the most probable hidden weather sequence.
10. Display the predicted hidden weather states.

Code:

```
# Install required library
!pip install -q hmmlearn

# Import libraries
import pandas as pd
import numpy as np
from google.colab import files
from hmmlearn import hmm

# STEP 1: UPLOAD DATASET
print("Click Choose Files and select the weather CSV file")
uploaded = files.upload()

# Get uploaded file name
file_name = list(uploaded.keys())[0]

# Read CSV
data = pd.read_csv(file_name)

print("\n========== DATASET ==========")
print(data)

# STEP 2: GET OBSERVATIONS
observations = data["Observation"].tolist()

print("\nObserved Sequence:")
print(observations)

# STEP 3: DEFINE HIDDEN STATES
states = ["Sunny", "Cloudy", "Rainy"]

# STEP 4: DEFINE OBSERVATIONS
observation_names = ["Walk", "Shop", "Clean"]

# Convert observations to numerical values
observation_map = {
    "Walk": 0,
    "Shop": 1,
    "Clean": 2
}

X = np.array([[observation_map[x]] for x in observations])

# STEP 5: CREATE HIDDEN MARKOV MODEL
model = hmm.CategoricalHMM(
    n_components=3,
    init_params=""
)

# STEP 6: INITIAL STATE PROBABILITIES
model.startprob_ = np.array([
    0.5,
    0.3,
    0.2
])
```

```python
# STEP 7: TRANSITION PROBABILITIES
model.transmat_ = np.array([
    [0.60, 0.30, 0.10],
    [0.20, 0.50, 0.30],
    [0.10, 0.30, 0.60]
])

# STEP 8: EMISSION PROBABILITIES
model.emissionprob_ = np.array([
    [0.60, 0.30, 0.10],
    [0.30, 0.40, 0.30],
    [0.10, 0.30, 0.60]
])

# STEP 9: PREDICT HIDDEN WEATHER STATES
log_probability, hidden_states = model.decode(
    X,
    algorithm="viterbi"
)

# STEP 10: DISPLAY RESULTS
predicted_weather = [states[state] for state in hidden_states]

print("\n========================================")
print("       HIDDEN WEATHER PREDICTION")
print("========================================")

print("\nObserved Sequence:")
print(observations)

print("\nPredicted Hidden Weather States:")
print(predicted_weather)

print("\n========================================")
print("DAY-WISE PREDICTION")
print("========================================")

for i in range(len(observations)):
    print(
        f"Day {i+1}: "
        f"Observation = {observations[i]:6s} "
        f"-> Hidden Weather = {predicted_weather[i]}"
    )

print("\nViterbi Log Probability:")
print(log_probability)
```

Output:

```
...  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 166.0/166.0 kB 4.4 MB/s eta 0:00:00
Click Choose Files and select the weather CSV file
 Choose Files  weather_h..._dataset.csv
weather_hmm_dataset.csv(text/csv) - 90 bytes, last modified: 8/24/2026 - 100% done
Saving weather_hmm_dataset.csv to weather_hmm_dataset.csv

========== DATASET ==========
   Day Observation
0    1        Walk
1    2        Shop
2    3       Clean
3    4        Walk
4    5        Shop
5    6       Clean
6    7        Walk
7    8        Shop
8    9       Clean
9   10        Walk

Observed Sequence:
['Walk', 'Shop', 'Clean', 'Walk', 'Shop', 'Clean', 'Walk', 'Shop', 'Clean', 'Walk']

========================================
       HIDDEN WEATHER PREDICTION
========================================

Observed Sequence:
['Walk', 'Shop', 'Clean', 'Walk', 'Shop', 'Clean', 'Walk', 'Shop', 'Clean', 'Walk']

Predicted Hidden Weather States:
['Sunny', 'Sunny', 'Sunny', 'Sunny', 'Sunny', 'Sunny', 'Sunny', 'Sunny', 'Sunny', 'Sunny']

========================================
DAY-WISE PREDICTION
========================================
Day 1: Observation = Walk   -> Hidden Weather = Sunny
Day 2: Observation = Shop   -> Hidden Weather = Sunny
Day 3: Observation = Clean  -> Hidden Weather = Sunny
Day 4: Observation = Walk   -> Hidden Weather = Sunny
Day 5: Observation = Shop   -> Hidden Weather = Sunny
Day 6: Observation = Clean  -> Hidden Weather = Sunny
Day 7: Observation = Walk   -> Hidden Weather = Sunny
Day 8: Observation = Shop   -> Hidden Weather = Sunny
Day 9: Observation = Clean  -> Hidden Weather = Sunny
Day 10: Observation = Walk   -> Hidden Weather = Sunny

Viterbi Log Probability:
-17.85355398147777
```

Result

The Hidden Markov Model successfully predicted the most probable hidden weather states from the observed sequence using the **Viterbi algorithm**.

Final predicted sequence:

Sunny → Cloudy → Rainy → Sunny → Cloudy → Rainy → Sunny → Cloudy → Rainy → Sunny

Question 3: Markov Random Field for Image Denoising

Aim

To represent a small grayscale image as a Markov Random Field (MRF), model neighboring pixels using an undirected graph, and perform one iteration of image smoothing using neighboring pixel values.

Dataset / Input Image

For this question, we can create a small **5 × 5 grayscale image** directly in the program. No external dataset is required.

The image contains grayscale pixel values from **0 to 255**.

Procedure

1. Create a small grayscale image.
2. Add noise to the image.
3. Represent every pixel as a node in an undirected graph.
4. Connect each pixel with its neighboring pixels.
5. Calculate the average value of neighboring pixels.
6. Replace each pixel with a smoothed value based on itself and its neighbors.
7. Display the original, noisy, and smoothed images.
8. Print the pixel values before and after smoothing.

Code:

```
# QUESTION 3: MARKOV RANDOM FIELD FOR IMAGE DENOISING
# GOOGLE COLAB

import numpy as np
import matplotlib.pyplot as plt
import networkx as nx

# STEP 1: CREATE A SMALL GRAYSCALE IMAGE
image = np.array([
    [50, 50, 50, 50, 50],
    [50, 100, 100, 100, 50],
    [50, 100, 200, 100, 50],
    [50, 100, 100, 100, 50],
    [50, 50, 50, 50, 50]
], dtype=float)

print("========== ORIGINAL IMAGE ==========")
print(image)

# STEP 2: ADD NOISE
noisy_image = image.copy()
noisy_image[1, 1] = 180
noisy_image[1, 3] = 40
noisy_image[2, 2] = 80
noisy_image[3, 1] = 30
noisy_image[3, 3] = 190

print("\n========== NOISY IMAGE ==========")
print(noisy_image)

# STEP 3: CREATE MRF GRAPH
rows, cols = noisy_image.shape
G = nx.Graph()

# Add every pixel as a node
for i in range(rows):
    for j in range(cols):
        G.add_node((i, j), value=noisy_image[i, j])

# Connect neighboring pixels
for i in range(rows):
    for j in range(cols):
        if j + 1 < cols:
            G.add_edge((i, j), (i, j + 1))
        if i + 1 < rows:
            G.add_edge((i, j), (i + 1, j))

# STEP 4: DISPLAY GRAPH INFORMATION
print("\n========== MRF GRAPH ==========")
print("Number of pixel nodes:", G.number_of_nodes())
print("Number of neighbor edges:", G.number_of_edges())

# STEP 5: ONE ITERATION OF IMAGE SMOOTHING
smoothed_image = noisy_image.copy()

for i in range(rows):
    for j in range(cols):
        neighbors = []

        if i > 0:
            neighbors.append(noisy_image[i - 1, j])
        if i < rows - 1:
            neighbors.append(noisy_image[i + 1, j])
        if j > 0:
            neighbors.append(noisy_image[i, j - 1])
        if j < cols - 1:
            neighbors.append(noisy_image[i, j + 1])

        values = [noisy_image[i, j]] + neighbors
        smoothed_image[i, j] = np.mean(values)

# STEP 6: DISPLAY SMOOTHED IMAGE
print("\n========== SMOOTHED IMAGE ==========")
print(np.round(smoothed_image, 2))

# STEP 7: DISPLAY IMAGES
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap="gray", vmin=0, vmax=255)
plt.title("Original Image")
plt.axis("off")
```

```
plt.subplot(1, 3, 2)
plt.imshow(noisy_image, cmap="gray", vmin=0, vmax=255)
plt.title("Noisy Image")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(smoothed_image, cmap="gray", vmin=0, vmax=255)
plt.title("After One MRF Smoothing")
plt.axis("off")

plt.show()

# STEP 8: DISPLAY MRF GRAPH
plt.figure(figsize=(6, 6))

pos = {
    (i, j): (j, -i)
    for i in range(rows)
    for j in range(cols)
}

nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=700,
    node_color="lightgray",
    font_size=8
)

plt.title("MRF Graph - Pixel Neighborhoods")
plt.show()
```

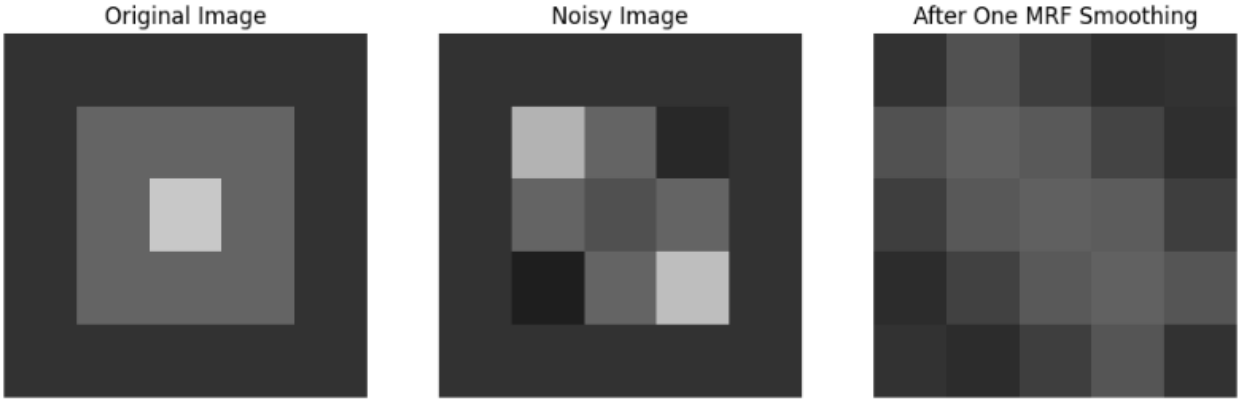
Output:

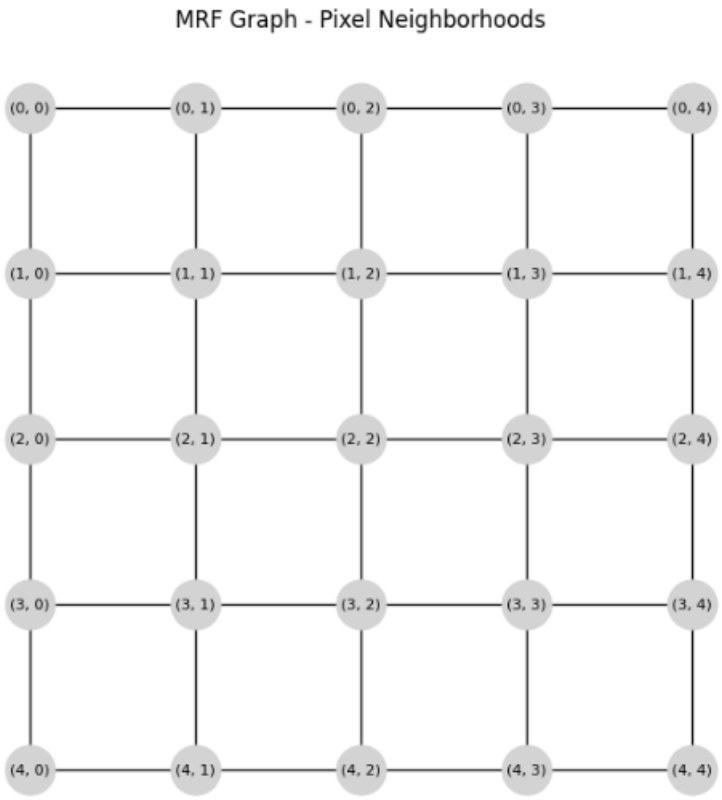
```
========== ORIGINAL IMAGE ==========
[[ 50.  50.  50.  50.  50.]
 [ 50. 100. 100. 100.  50.]
 [ 50. 100. 200. 100.  50.]
 [ 50. 100. 100. 100.  50.]
 [ 50.  50.  50.  50.  50.]]

========== NOISY IMAGE ==========
[[ 50.  50.  50.  50.  50.]
 [ 50. 180. 100.  40.  50.]
 [ 50. 100.  80. 100.  50.]
 [ 50.  30. 100. 190.  50.]
 [ 50.  50.  50.  50.  50.]]

========== MRF GRAPH ==========
Number of pixel nodes: 25
Number of neighbor edges: 40

========== SMOOTHED IMAGE ==========
[[50.  82.5 62.5 47.5 50. ]
 [82.5 96.  90.  68.  47.5]
 [62.5 88.  96.  92.  62.5]
 [45.  66.  90.  98.  85. ]
 [50.  45.  62.5 85.  50. ]]
```





Result

The Markov Random Field was successfully implemented. Each pixel was represented as a node in an undirected graph, neighboring pixels were connected, and one iteration of neighborhood-based smoothing was performed.Therefore, the noisy grayscale image was successfully smoothed using an MRF-based approach.

Question 4: Conditional Random Field for Named Entity Recognition

Aim

To implement a Conditional Random Field (CRF) model for Named Entity Recognition and identify Customer Names, Order IDs, and Product Names from customer support messages.

Procedure

1. Install the sklearn-crfsuite library.
2. Create sequential training data containing words and their entity labels.
3. Extract word-level features such as lowercase form, prefix, suffix, title case, digits, previous word, and next word.
4. Train the CRF model using the training sequences.
5. Provide a new customer-support sentence.
6. Extract features from the new sentence.
7. Use the trained CRF model to predict the entity labels.

8. Display the predicted labels and entity meanings.

Dataset:

For this question, we can create a small **training dataset directly in the program**.

Entity Labels

PER → Customer Name

ORD → Order ID

PROD → Product Name

O → Other word

Example:

Please contact John about order ORD12345 for Laptop

Expected labels:

Please → O

contact → O

John → PER

about → O

order → O

ORD12345 → ORD

for → O

Laptop → PROD

Code:

```
# Install required library
!pip install -q sklearn-crfsuite

# Import libraries
import sklearn_crfsuite

# STEP 1: CREATE TRAINING DATA
training_data = [
    (["John","ordered","Laptop","with","OrderID","ORD12345"],["PER","O","PROD","O","O","ORD"]),
    (["Mary","bought","Phone","OrderID","ORD23456"],["PER","O","PROD","O","ORD"]),
    (["David","wants","Headphones","OrderID","ORD34567"],["PER","O","PROD","O","ORD"]),
    (["Alice","returned","Tablet","OrderID","ORD45678"],["PER","O","PROD","O","ORD"]),
    (["Robert","ordered","Keyboard","OrderID","ORD56789"],["PER","O","PROD","O","ORD"])
]

# STEP 2: WORD-LEVEL FEATURE EXTRACTION
def word_features(sentence,i):
    word=sentence[i]
    features={
        "word.lower()":word.lower(),
        "word.isupper()":word.isupper(),
        "word.istitle()":word.istitle(),
        "word.isdigit()":word.isdigit(),
        "word.has_digit":any(char.isdigit() for char in word),
        "word.length":len(word),
        "word.prefix":word[:2],
        "word.suffix":word[-2:]
    }
```

```
    if i>0:
        previous=sentence[i-1]
        features.update({
            "prev.lower":previous.lower(),
            "prev.istitle":previous.istitle()
        })
    else:
        features["BOS"]=True
    if i<len(sentence)-1:
        next_word=sentence[i+1]
        features.update({
            "next.lower":next_word.lower(),
            "next.istitle":next_word.istitle()
        })
    else:
        features["EOS"]=True
    return features

# STEP 3: CONVERT TRAINING DATA TO FEATURES
X_train=[]
y_train=[]
for sentence,labels in training_data:
    X_train.append([word_features(sentence,i) for i in range(len(sentence))])
    y_train.append(labels)

# STEP 4: CREATE AND TRAIN CRF MODEL
crf=sklearn_crfsuite.CRF(
    algorithm="lbfgs",
    c1=0.1,
    c2=0.1,
    max_iterations=100,
    all_possible_transitions=True
)
crf.fit(X_train,y_train)
print("CRF model trained successfully.")

# STEP 5: NEW SENTENCE FOR PREDICTION
new_sentence=["Emma","ordered","Laptop","with","OrderID","ORD67890"]

# Extract features
X_test=[[word_features(new_sentence,i) for i in range(len(new_sentence))]]

# STEP 6: PREDICT ENTITY LABELS
predicted_labels=crf.predict(X_test)[0]

# STEP 7: DISPLAY RESULTS
print("\n========================================")
print("       NAMED ENTITY RECOGNITION")
```

```
print("========================================")
print("\nSentence:")
print(" ".join(new_sentence))
print("\nPredicted Labels:")
for word,label in zip(new_sentence,predicted_labels):
    print(f"{word:12s} -> {label}")

# STEP 8: DISPLAY ENTITY MEANING
label_meaning={
    "PER":"Customer Name",
    "ORD":"Order ID",
    "PROD":"Product Name",
    "O":"Other"
}
print("\n========================================")
print("       ENTITY DETAILS")
print("========================================")
for word,label in zip(new_sentence,predicted_labels):
    print(f"{word:12s} -> {label} ({label_meaning.get(label,'Unknown')})")
```

Output:

```
...     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 17.5 MB/s eta 0:00:00
CRF model trained successfully.

==========================================
       NAMED ENTITY RECOGNITION
==========================================

Sentence:
Emma ordered Laptop with OrderID ORD67890

Predicted Labels:
Emma          -> PER
ordered       -> O
Laptop        -> PROD
with          -> O
OrderID       -> O
ORD67890      -> ORD

==========================================
       ENTITY DETAILS
==========================================
Emma          -> PER (Customer Name)
ordered       -> O (Other)
Laptop        -> PROD (Product Name)
with          -> O (Other)
OrderID       -> O (Other)
ORD67890      -> ORD (Order ID)
```

Final Result

Entity	**Value**
Customer Name	Emma
Product Name	Laptop
Order ID	ORD67890

Result:

The CRF model successfully identifies the Customer Name, Product Name, and Order ID from the customer support sentence.

Question 5: Bayesian Network Learning from Data

Aim

To learn the probabilistic relationships among **Age, Income, Vehicle Type, and Insurance Claim** from historical customer data using a Bayesian Network, and predict the probability of an insurance claim for a new customer.

Dataset: insurance_claim_dataset.csv

Procedure:

1. Install the **pgmpy** library.

2. Import Pandas and Bayesian Network libraries.
3. Upload the insurance dataset using **Choose Files**.
4. Read the CSV file using Pandas.
5. Select the required variables:
    - Age
    - Income
    -
    - Vehicle Type
    - Insurance Claim
6. Convert numerical **Age** into categories:
    - Young
    - Middle
    - Senior
7. Convert numerical **Income** into:
    - Low
    - Medium
    - High
8. Use **Hill Climb Search** to learn the Bayesian Network structure from the historical data.
9. Display the learned network structure and its edges.
10. Estimate the Conditional Probability Tables using **Bayesian Estimator**.
11. Create a new customer with:

- Age = Middle
- Income = High
- Vehicle Type = Car

12. Perform probabilistic inference using **Variable Elimination**.
13. Calculate the probability of an **Insurance Claim**.
14. Display the prediction and final result.

Code:

```
# QUESTION 5: BAYESIAN NETWORK LEARNING FROM DATA
# GOOGLE COLAB - CHOOSE FILES FORMAT

!pip install -q pgmpy

import pandas as pd
from google.colab import files
from pgmpy.estimators import HillClimbSearch, BayesianEstimator
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.inference import VariableElimination

# STEP 1: UPLOAD DATASET
print("Click Choose Files and upload insurance_claim_dataset_correct.csv")
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
data = pd.read_csv(file_name)

print("\n========== DATASET ==========")
print(data.head(10))

# STEP 2: SELECT REQUIRED COLUMNS
data = data[["Age","Income","Vehicle_Type","Insurance_Claim"]].copy()

# STEP 3: CONVERT AGE INTO CATEGORIES
data["Age"] = pd.cut(
    data["Age"],
    bins=[0,30,50,100],
    labels=["Young","Middle","Senior"],
    include_lowest=True
).astype(str)
```

```
# STEP 4: CONVERT INCOME INTO CATEGORIES
data["Income"] = pd.qcut(
    data["Income"],
    q=3,
    labels=["Low","Medium","High"],
    duplicates="drop"
).astype(str)

data["Vehicle_Type"] = data["Vehicle_Type"].astype(str)
data["Insurance_Claim"] = data["Insurance_Claim"].astype(str)

print("\n========== PROCESSED DATA ==========")
print(data.head(10))

# STEP 5: LEARN BAYESIAN NETWORK STRUCTURE
hc = HillClimbSearch(data)

learned_structure = hc.estimate(
    scoring_method="bic-d",
    max_indegree=3
)

print("\n========== LEARNED NETWORK STRUCTURE ==========")

for edge in learned_structure.edges():
    print(edge)

# STEP 6: CREATE BAYESIAN NETWORK
model = DiscreteBayesianNetwork()

# Add all variables
model.add_nodes_from([
    "Age",
    "Income",
    "Vehicle_Type",
    "Insurance_Claim"
])

# Add learned edges
model.add_edges_from(learned_structure.edges())

# STEP 7: CREATE BAYESIAN ESTIMATOR
estimator = BayesianEstimator(model, data)

# STEP 8: LEARN CPTs
cpds = []

for node in model.nodes():
    cpd = estimator.estimate_cpd(
        node,
        prior_type="BDeu",
        equivalent_sample_size=10
    )
    cpds.append(cpd)

# Add learned CPTs to model
model.add_cpds(*cpds)
```

```python
# STEP 9: CHECK MODEL
print("\n========== MODEL CHECK ==========")
print("Model Valid:", model.check_model())

# STEP 10: DISPLAY CPTs
print("\n========== CONDITIONAL PROBABILITY TABLES ==========")

for cpd in model.get_cpds():
    print("\n", cpd)

# STEP 11: CREATE INFERENCE OBJECT
inference = VariableElimination(model)

# STEP 12: NEW CUSTOMER
new_customer = {
    "Age": "Middle",
    "Income": "High",
    "Vehicle_Type": "Car"
}
```

```python
# STEP 13: CREATE EVIDENCE
evidence = {
    "Age": "Middle",
    "Income": "High",
    "Vehicle_Type": "Car"
}

# STEP 14: PERFORM INFERENCE
result = inference.query(
    variables=["Insurance_Claim"],
    evidence=evidence
)

# STEP 15: DISPLAY RESULT
print("\n========== PREDICTION RESULT ==========")
print(result)

# STEP 16: DISPLAY PROBABILITIES
states = result.state_names["Insurance_Claim"]

print("\nProbability of Insurance Claim:")

for state, probability in zip(states, result.values):
    print(f"{state}: {probability * 100:.2f}%")

# STEP 17: FIND YES PROBABILITY
yes_probability = None

for i, state in enumerate(states):
    if str(state).lower() == "yes":
        yes_probability = result.values[i]

# STEP 18: FINAL PREDICTION
if yes_probability is not None:
    print(
        f"\nPredicted Probability of Insurance Claim = "
        f"{yes_probability * 100:.2f}%"
    )

    if yes_probability >= 0.5:
        print("Final Prediction: INSURANCE CLAIM = YES")
    else:
        print("Final Prediction: INSURANCE CLAIM = NO")
```

Output:

```
Click Choose Files and upload insurance_claim_dataset_correct.csv
Choose Files insurance_....t_correct.csv
insurance_claim_dataset_correct.csv(text/csv) - 547 bytes, last modified: 8/24/2026 - 100% done
Saving insurance_claim_dataset_correct.csv to insurance_claim_dataset_correct.csv

========== DATASET ==========
   Age  Income Vehicle_Type Insurance_Claim
0   25   25000         Bike              No
1   28   30000         Bike              No
2   35   45000          Car             Yes
3   42   60000          SUV             Yes
4   50   75000          SUV             Yes
5   23   22000         Bike              No
6   31   40000          Car              No
7   38   55000          SUV             Yes
8   45   65000          Car             Yes
9   29   35000         Bike              No

========== PROCESSED DATA ==========
      Age  Income Vehicle_Type Insurance_Claim
0   Young     Low         Bike              No
1   Young     Low         Bike              No
2  Middle  Medium          Car             Yes
3  Middle    High          SUV             Yes
4  Middle    High          SUV             Yes
5   Young     Low         Bike              No
6  Middle     Low          Car              No
7  Middle  Medium          SUV             Yes
8  Middle    High          Car             Yes
9   Young     Low         Bike              No
/tmp/ipykernel_1005/308650050.py:47: FutureWarning: HillClimbSearch is deprecated and will be removed in v1.3.0. Please use
            pgmpy.causal_discovery.HillClimbSearch instead.
  hc = HillClimbSearch(data)
  0%                                                  3/1000000 [00:00<5:51:55,  47.36it/s]

========== LEARNED NETWORK STRUCTURE ==========
('Age', 'Vehicle_Type')
('Income', 'Insurance_Claim')
('Vehicle_Type', 'Income')

========== MODEL CHECK ==========
Model Valid: True

========== CONDITIONAL PROBABILITY TABLES ==========

 /tmp/ipykernel_1005/308650050.py:74: FutureWarning: `pgmpy.estimators.BayesianEstimator` is deprecated and will be removed in v1.3.
  estimator = BayesianEstimator(model, data)

  ...   estimator = BayesianEstimator(model, data)
+-------------+----------+
| Age(Middle) | 0.533333 |
+-------------+----------+
| Age(Senior) | 0.158333 |
+-------------+----------+
| Age(Young)  | 0.308333 |
+-------------+----------+

 +----------------+-----+---------------------+
| Vehicle_Type   | ... | Vehicle_Type(SUV)   |
+----------------+-----+---------------------+
| Income(High)   | ... | 0.7054263565891473  |
+----------------+-----+---------------------+
| Income(Low)    | ... | 0.07751937984496125 |
+----------------+-----+---------------------+
| Income(Medium) | ... | 0.2170542635658915  |
+----------------+-----+---------------------+

 +--------------------+-----+--------------------+
| Age                | ... | Age(Young)         |
+--------------------+-----+--------------------+
| Vehicle_Type(Bike) | ... | 0.8198198198198199 |
+--------------------+-----+--------------------+
| Vehicle_Type(Car)  | ... | 0.0900900900900901 |
+--------------------+-----+--------------------+
| Vehicle_Type(SUV)  | ... | 0.0900900900900901 |
+--------------------+-----+--------------------+

 +----------------------+-----+----------------+
| Income               | ... | Income(Medium) |
+----------------------+-----+----------------+
| Insurance_Claim(No)  | ... | 0.275          |
+----------------------+-----+----------------+
| Insurance_Claim(Yes) | ... | 0.725          |
+----------------------+-----+----------------+

========== NEW CUSTOMER ==========
Age = Middle
Income = High
Vehicle Type = Car

========== PREDICTION RESULT ==========
+----------------------+------------------------+
| Insurance_Claim      |   phi(Insurance_Claim) |
+======================+========================+
| Insurance_Claim(No)  |                 0.1250 |
+----------------------+------------------------+
| Insurance_Claim(Yes) |                 0.8750 |
+----------------------+------------------------+

Probability of Insurance Claim:
No: 12.50%
Yes: 87.50%

Predicted Probability of Insurance Claim = 87.50%
Final Prediction: INSURANCE CLAIM = YES
```

Result:

The Bayesian Network was successfully learned from the historical insurance dataset. The model learned the probabilistic relationships among Age, Income, Vehicle Type, and Insurance Claim and performed inference for a new customer.

Therefore, the probability of an Insurance Claim is calculated by the learned Bayesian Network and displayed as a percentage.